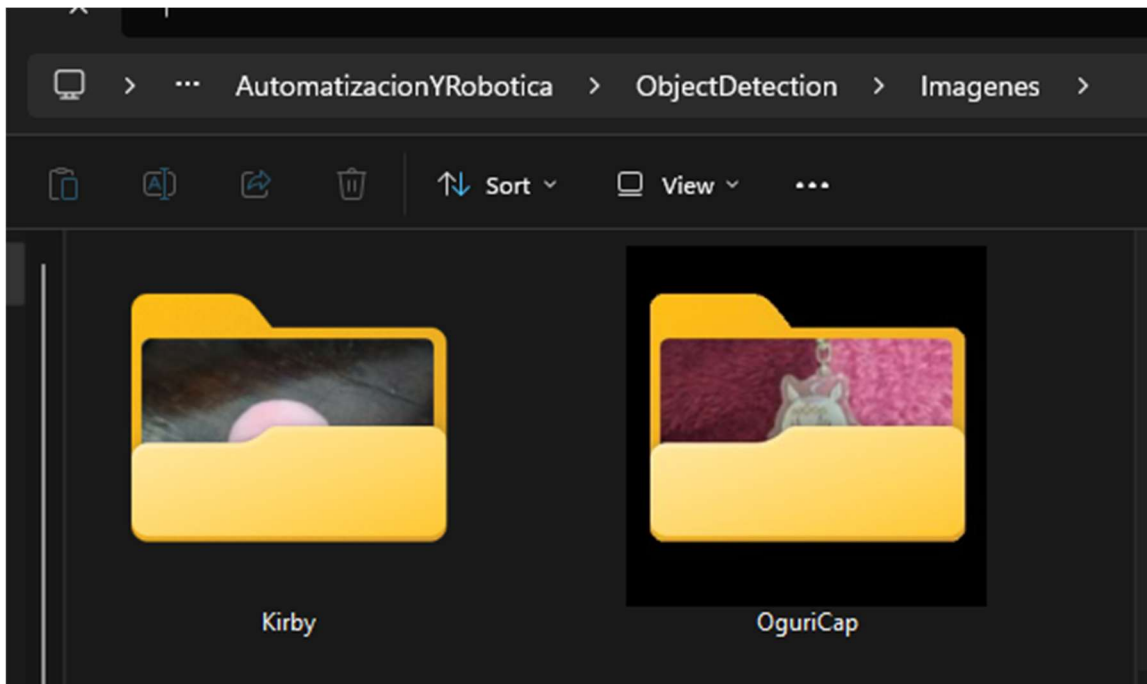


Recolección de imágenes.

Primero se necesitan juntar una extensa cantidad de imágenes de lo que se desea clasificar, por lo menos 30 por clase/objeto para que se detectar con una consistencia notable. Recordar, que es necesario que las imágenes tengan variabilidad (posición del objeto, fondo diferente, o iluminación) para que el entrenamiento del modelo sea correcto.

Para cada objeto/clase, crear una carpeta con las imágenes de ese objeto, y después poner todas las carpetas en una carpeta.



Programación

Este proyecto se realizó en **Google Colab**, porque la librería **mediapipe-model-maker** fallaba en los IDEs de Python por alguna razón.

Primero se comprime en un .zip la carpeta con todas las imágenes y se suben al colab.

```
!sudo apt-get install python3.10 python3.10-distutils -y
!wget https://bootstrap.pypa.io/get-pip.py
!python3.10 get-pip.py
!python3.10 -m pip install -q mediapipe-model-maker
```

```
!unzip -o -q Imagenes.zip -d /content/dataset
```

Este bloque de código, genera la versión de Python (≤ 3.10) para que pueda usar la librería de **mediapipe-model-maker**. Posteriormente extraer el contenido de las imágenes (Imagenes.zip) en una carpeta dentro de **content/dataset**. Procurar que el nombre en el unzip sea el mismo que el que se importó.

Posteriormente se corre el siguiente bloque de código:

```
%%writefile entrenar.py
import os
from mediapipe_model_maker import image_classifier

DATASET_PATH = "/content/dataset/Imagenes"
data = image_classifier.Dataset.from_folder(DATASET_PATH)
train_data, validation_data = data.split(0.8)

print(f"Imágenes para entrenar: {len(train_data)}")
print(f"Imágenes para validar: {len(validation_data)}")

# --- LA CORRECCIÓN ESTÁ AQUÍ ---
hparams = image_classifier.HParams(epochs=15)
options = image_classifier.ImageClassifierOptions(
    supported_model=image_classifier.SupportedModels.EFFICIENTNET_LITE0,
    hparams=hparams
)
# -----

print("Iniciando entrenamiento...")
model = image_classifier.ImageClassifier.create(
    train_data=train_data,
    validation_data=validation_data,
    options=options,
)

loss, accuracy = model.evaluate(validation_data)
print(f"Precisión final: {accuracy * 100:.2f}%")

model.export_model("modelo_nuevo.tflite")
print("¡Modelo exportado con éxito!")
```

Este bloque de código, creara un .py que dividirá las imágenes en entrenamiento y validación y con eso generara el modelo.

El próximo código, corre el código recién creado con la instancia de Python creada al inicio.

```
!MPLBACKEND=agg python3.10 entrenar.py
```

De ahí, probablemente el modelo **modelo_nuevo.tflite** se encuentre perdido entre todas las carpetas del Google Colab, entonces corremos este bloque de código para que la encuentre y la descargue.

```
import subprocess
from google.colab import files

print("Escaneando la totalidad del servidor (esto tomará unos segundos)...")

# Ejecutamos el comando 'find' de Linux desde la raíz absoluta '/'
resultado = subprocess.run(['find', '/', '-name', '*.tflite', '-type', 'f'], capture_output=True,
text=True)

# Limpiamos los resultados
rutas = resultado.stdout.strip().split('\n')

# Filtramos para ignorar los modelos de ejemplo que vienen instalados dentro de la
propia librería de MediaPipe
archivos_validos = [r for r in rutas if r and "site-packages" not in r and "dist-packages" not
in r]

if archivos_validos:
    # Tomamos el archivo más reciente o el primero que encuentre
    ruta_exacta = archivos_validos[-1]
    print(f"¡BINGO! Tu modelo estaba escondido en: {ruta_exacta}")
    print("Iniciando la descarga a tu computadora...")

    # Forzar descarga
    files.download(ruta_exacta)
else:
    print("El escáner terminó. Definitivamente no hay ningún modelo generado en el
servidor.")
    print("Si sale este mensaje, significa que el script 'entrenar.py' no logró ejecutar la
última línea de exportación.")
```